

Schema Layer

Schema Layer

```
/* CREATE DATABASE Insurance_ETL_CaseStudy;
GO
USE Insurance_ETL_CaseStudy;
GO
*/

-- Table STG Policy
-- Here in this table SRC_UPDATE_TS Column Required to execute the "keep the latest" deduplication logic.

/* CREATE TABLE dbo.STG_POLICY (
    POL_NBR    VARCHAR(20)    NULL,
    EFF_DT     VARCHAR(8) NULL,
    PREM_AMT   DECIMAL(12,2) NULL,
    ST_CD      CHAR(2)       NULL,
    LOB_CD     VARCHAR(10)    NULL,
    LNAME      VARCHAR(40)    NULL,
    FNAME      VARCHAR(40)    NULL,
    CANCEL_FLG CHAR(1)        NULL,
    SRC_UPDATE_TS DATETIME2    NULL
);
*/

/*TRUNCATE TABLE dbo.STG_POLICY;
GO

INSERT INTO dbo.STG_POLICY
    (POL_NBR, EFF_DT, PREM_AMT, ST_CD, LOB_CD, LNAME, FNAME, CANCEL_FLG, SRC_UPDATE_TS)
VALUES
    -- =====
    -- 1(Perfectly clean data that should pass effortlessly)
    -- =====
    ('POL-10001', '20231001', 1200.50, 'TX', 'AUTO', 'Smith', 'John', 'N', '2023-10-01 08:00:00'),
    ('POL-10002', '20231002', 850.00, 'CA', 'HOME', 'Johnson', 'Mary', 'N', '2023-10-02 08:15:00'),
    ('POL-10003', '20231003', 400.25, 'NY', 'RENTER', 'Williams', 'James', 'N', '2023-10-03 09:30:00'),
    ('POL-10004', '20231004', 2100.00, 'FL', 'AUTO', 'Brown', 'Patricia', 'Y', '2023-10-04 10:45:00'),
    ('POL-10005', '20231005', 999.99, 'IL', 'HOME', 'Jones', 'Robert', 'N', '2023-10-05 11:20:00'),
    ('POL-10006', '20231006', 350.00, 'TX', 'RENTER', 'Garcia', 'Linda', 'Y', '2023-10-06 13:00:00'),
    ('POL-10007', '20231007', 1750.75, 'CA', 'AUTO', 'Miller', 'Michael', 'N', '2023-10-07 14:10:00'),
    ('POL-10008', '20231008', 1100.00, 'NY', 'HOME', 'Davis', 'Barbara', 'N', '2023-10-08 15:55:00'),
    ('POL-10009', '20231009', 200.00, 'FL', 'RENTER', 'Rodriguez', 'William', 'N', '2023-10-09 16:30:00'),
    ('POL-10010', '20231010', 3200.00, 'IL', 'AUTO', 'Martinez', 'Richard', 'Y', '2023-10-10 17:45:00'),

    -- =====
    -- 2. DEDUPLICATION TESTS (Testing the Grain: "Keep the latest by SRC_UPDATE_TS")
    -- =====
    -- Policy 20001 arrives 3 times. Only the 10:05 record should survive.
    ('POL-20001', '20231101', 1000.00, 'TX', 'AUTO', 'Hernandez', 'Joseph', 'N', '2023-11-01 08:00:00'),
```

Schema Layer

```
('POL-20001', '20231101', 1050.00, 'TX', 'AUTO', 'Hernandez', 'Joseph', 'N', '2023-11-01 09:30:00'),
('POL-20001', '20231101', 1100.00, 'TX', 'AUTO', 'Hernandez', 'Joseph', 'N', '2023-11-01 10:05:00'),
-- Policy 20002 arrives 2 times.
('POL-20002', '20231102', 600.00, 'CA', 'HOME', 'Lopez', 'Charles', 'N', '2023-11-02 11:00:00'),
('POL-20002', '20231102', 600.00, 'CA', 'HOME', 'Lopez', 'Charles', 'Y', '2023-11-02 14:00:00'),

-- =====
-- 3. DATE CASTING ERRORS (These should route to the Error Table)
-- =====
('POL-30001', '20230229', 500.00, 'NY', 'AUTO', 'Gonzalez', 'Thomas', 'N', '2023-11-05 08:00:00'), -- Invalid
date (Not a leap year)
('POL-30002', '20231301', 450.00, 'FL', 'HOME', 'Wilson', 'Sarah', 'N', '2023-11-05 09:00:00'), -- Invalid
month (13)
('POL-30003', 'ABCDEFGH', 300.00, 'TX', 'RENTER', 'Anderson', 'Matthew', 'N', '2023-11-05 10:00:00'), --
Pure garbage string
('POL-30004', '', 700.00, 'CA', 'AUTO', 'Thomas', 'Karen', 'N', '2023-11-05 11:00:00'), -- Empty string
('POL-30005', NULL, 800.00, 'IL', 'HOME', 'Taylor', 'Nancy', 'N', '2023-11-05 12:00:00'), -- NULL
date

-- =====
-- 4. STATE CODE TESTS (Lookup failures & Case formatting)
-- =====
('POL-40001', '20231110', 1200.00, 'tx', 'AUTO', 'Moore', 'Lisa', 'N', '2023-11-10 08:00:00'), -- Lowercase
(Should be upppered and PASS)
('POL-40002', '20231110', 1300.00, 'ZZ', 'HOME', 'Jackson', 'Betty', 'N', '2023-11-10 09:00:00'), -- Invalid
State (Should FAIL to error table)
('POL-40003', '20231110', 1400.00, NULL, 'RENTER', 'Martin', 'Margaret', 'N', '2023-11-10 10:00:00'), -- NULL
State (Should FAIL)
('POL-40004', '20231110', 1500.00, ' C', 'AUTO', 'Lee', 'Sandra', 'N', '2023-11-10 11:00:00'), -- Bad
padding

-- =====
-- 5. LOB CODE TESTS (Default to 'UNKNOWN' if no match)
-- =====
('POL-50001', '20231115', 900.00, 'TX', 'UMBRELLA', 'Perez', 'Ashley', 'N', '2023-11-15 08:00:00'), -- Valid
syntax, but 'UMBRELLA' won't be in REF_LOB
('POL-50002', '20231115', 950.00, 'CA', NULL, 'Thompson', 'Steven', 'N', '2023-11-15 09:00:00'), -- NULL
LOB

-- =====
-- 6. PREMIUM AMOUNT TESTS (Default to 0.00 if NULL)
-- =====
('POL-60001', '20231120', NULL, 'NY', 'AUTO', 'White', 'Paul', 'N', '2023-11-20 08:00:00'), -- NULL
Premium
('POL-60002', '20231120', NULL, 'FL', 'HOME', 'Harris', 'Andrew', 'N', '2023-11-20 09:00:00'), -- NULL
Premium

-- =====
```

Schema Layer

```
-- 7. NAME CONCATENATION TESTS (Handling NULLs safely)
-- =====
('POL-70001', '20231125', 400.00, 'TX', 'RENTER', NULL,      'Joshua', 'N', '2023-11-25 08:00:00'), --
Missing Last Name
('POL-70002', '20231125', 450.00, 'CA', 'AUTO', 'Sanchez', NULL,      'N', '2023-11-25 09:00:00'), --
Missing First Name
('POL-70003', '20231125', 500.00, 'NY', 'HOME', NULL,      NULL,      'N', '2023-11-25 10:00:00'), --
Missing Both

-- =====
-- 8. STRING PADDING & STATUS TESTS (Trim policies, decode statuses)
-- =====
(' POL-80001 ', '20231201', 100.00, 'FL', 'AUTO', 'Clark', 'Kenneth', 'Y', '2023-12-01 08:00:00'), --
Leading/Trailing spaces on Policy Number
('POL-80002', '20231201', 150.00, 'TX', 'HOME', 'Ramirez', 'Kevin', 'y', '2023-12-01 09:00:00'), --
Lowercase 'y'
('POL-80003', '20231201', 200.00, 'CA', 'RENTER', 'Lewis', 'Brian', 'X', '2023-12-01 10:00:00'), -- Invalid
Status flag (Should map to 'UNKNOWN')
('POL-80004', '20231201', 250.00, 'NY', 'AUTO', 'Robinson', 'George', NULL, '2023-12-01 11:00:00'), -- NULL
Status flag (Should map to 'UNKNOWN')

-- =====
-- =====
('POL-90001', '20231210', 750.00, 'FL', 'HOME', 'Walker', 'Edward', 'N', '2023-12-10 08:00:00'),
('POL-90002', '20231210', 800.00, 'TX', 'AUTO', 'Young', 'Ronald', 'N', '2023-12-10 09:00:00'),
('POL-90003', '20231210', 850.00, 'CA', 'RENTER', 'Allen', 'Timothy', 'Y', '2023-12-10 10:00:00'),
('POL-90004', '20231210', 900.00, 'NY', 'HOME', 'King', 'Jason', 'N', '2023-12-10 11:00:00'),
('POL-90005', '20231210', 950.00, 'FL', 'AUTO', 'Wright', 'Jeffrey', 'N', '2023-12-10 12:00:00'),
('POL-90006', '20231210', 1000.00, 'TX', 'HOME', 'Scott', 'Ryan', 'N', '2023-12-10 13:00:00'),
('POL-90007', '20231210', 1050.00, 'CA', 'AUTO', 'Torres', 'Jacob', 'Y', '2023-12-10 14:00:00'),
('POL-90008', '20231210', 1100.00, 'NY', 'RENTER', 'Nguyen', 'Gary', 'N', '2023-12-10 15:00:00'),
('POL-90009', '20231210', 1150.00, 'FL', 'HOME', 'Hill', 'Nicholas', 'N', '2023-12-10 16:00:00'),
('POL-90010', '20231210', 1200.00, 'TX', 'AUTO', 'Flores', 'Eric', 'N', '2023-12-10 17:00:00'),
('POL-90011', '20231210', 1250.00, 'CA', 'HOME', 'Green', 'Stephen', 'N', '2023-12-10 18:00:00'),
('POL-90012', '20231210', 1300.00, 'NY', 'AUTO', 'Adams', 'Jonathan', 'Y', '2023-12-10 19:00:00');

GO
*/

-- SELECT * FROM dbo.STG_POLICY;
```

Schema Layer

KROCKZI\SQLPRESS | @Insurance_ETL_CaseStudy

1Select * from [dbo].[STG_POLICY];

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	QUERY RESULTS	AZURE												
Results (1) Messages																		
	POL_NBR	↕ ↕	EFF_DT	↕ ↕	PREM_AMT	↕ ↕	ST_CD	↕ ↕	LOB_CD	↕ ↕	LNAME	↕ ↕	FNAME	↕ ↕	CANCEL_FLG	↕ ↕	SRC_UPDATE_TS	↕ ↕
1	POL-10001		20231001		1500.00		TX		AUTO		Smith		John		N		2023-06-22 02:54:53.0266667	
2	POL-10002		20231002		850.00		CA		HOME		Johnson		Mary		N		2023-10-02 08:15:00.0000000	
3	POL-10003		20231003		400.25		NY		RENTER		Williams		James		N		2023-10-03 09:30:00.0000000	
4	POL-10004		20231004		2100.00		FL		AUTO		Brown		Patricia		Y		2023-10-04 10:45:00.0000000	
5	POL-10005		20231005		999.99		IL		HOME		Jones		Robert		N		2023-10-05 11:20:00.0000000	
6	POL-10006		20231006		350.00		TX		RENTER		Garcia		Linda		Y		2023-10-06 13:00:00.0000000	
7	POL-10007		20231007		1750.75		CA		AUTO		Miller		Michael		N		2023-10-07 14:10:00.0000000	
8	POL-10008		20231008		1100.00		NY		HOME		Davis		Barbara		N		2023-10-08 15:55:00.0000000	
9	POL-10009		20231009		200.00		FL		RENTER		Rodriguez		William		N		2023-10-09 16:30:00.0000000	
10	POL-10010		20231010		3200.00		IL		AUTO		Martinez		Richard		Y		2023-10-10 17:45:00.0000000	
11	POL-20001		20231101		1000.00		TX		AUTO		Hernandez		Joseph		N		2023-11-01 08:00:00.0000000	
12	POL-20001		20231101		1050.00		TX		AUTO		Hernandez		Joseph		N		2023-11-01 09:30:00.0000000	
13	POL-20001		20231101		1100.00		TX		AUTO		Hernandez		Joseph		N		2023-11-01 10:05:00.0000000	
14	POL-20002		20231102		600.00		CA		HOME		Lopez		Charles		N		2023-11-02 11:00:00.0000000	
15	POL-20002		20231102		600.00		CA		HOME		Lopez		Charles		Y		2023-11-02 14:00:00.0000000	
16	POL-30001		20230229		500.00		NY		AUTO		Gonzalez		Thomas		N		2023-11-05 08:00:00.0000000	
17	POL-30002		20231301		450.00		FL		HOME		Wilson		Sarah		N		2023-11-05 09:00:00.0000000	
18	POL-30003		ABCDEF GH		300.00		TX		RENTER		Anderson		Matthew		N		2023-11-05 10:00:00.0000000	
19	POL-30004				700.00		CA		AUTO		Thomas		Karen		N		2023-11-05 11:00:00.0000000	
20	POL-30005		NULL		800.00		IL		HOME		Taylor		Nancy		N		2023-11-05 12:00:00.0000000	

```

/*
=====
-- 1. STATE REFERENCE TABLE
=====
IF OBJECT_ID('dbo.REF_STATE', 'U') IS NOT NULL
    DROP TABLE dbo.REF_STATE;
GO

-- Using CHAR(2) for the Primary Key since state codes are strictly two letters
CREATE TABLE dbo.REF_STATE (
    STATE_CD CHAR(2) PRIMARY KEY
);
GO

-- Populating with valid states to test our STG data
INSERT INTO dbo.REF_STATE (STATE_CD)
VALUES ('TX'), ('CA'), ('NY'), ('FL'), ('IL');
GO

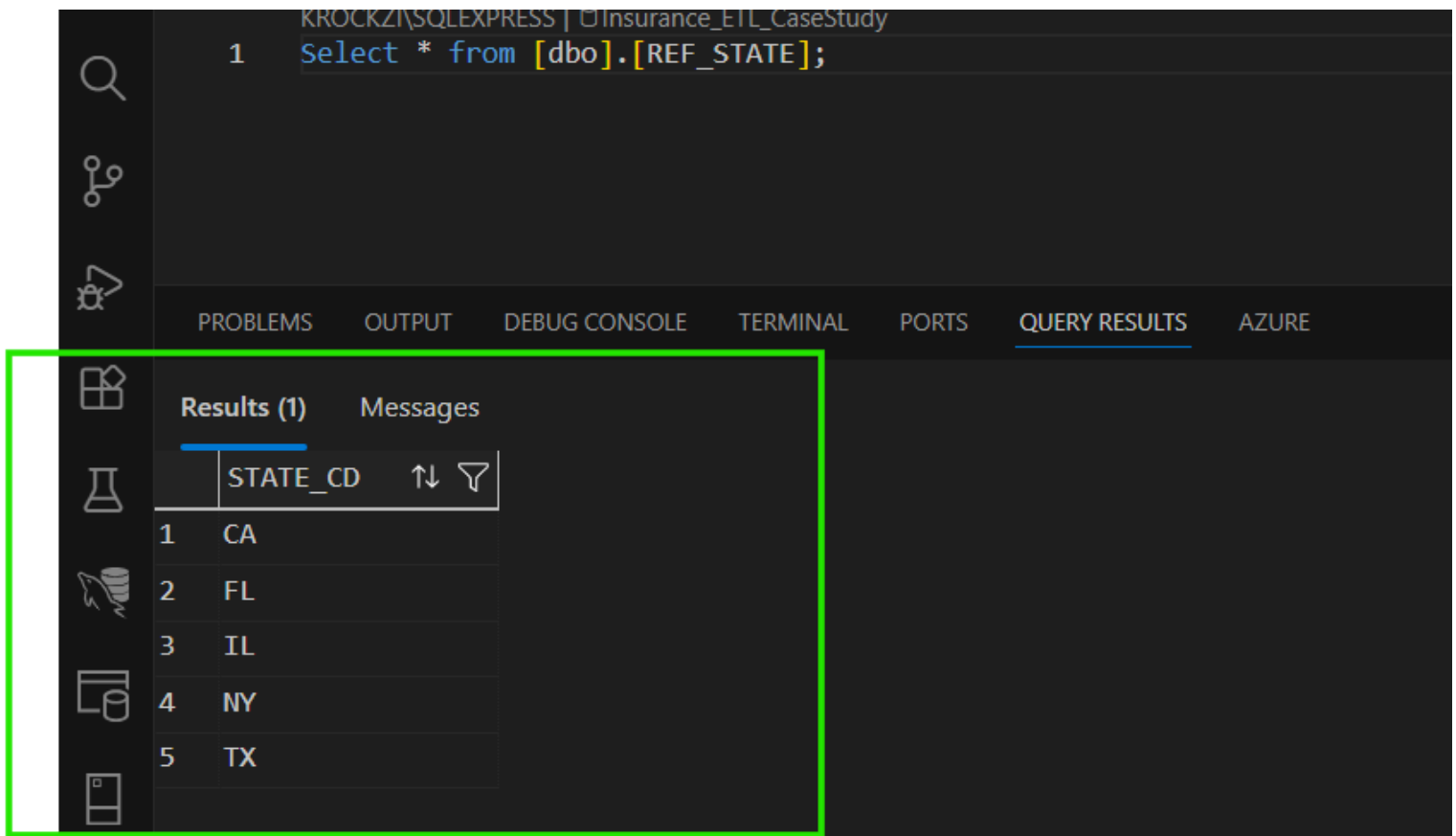
=====
-- 2. LINE OF BUSINESS (LOB) REFERENCE TABLE
=====
IF OBJECT_ID('dbo.REF_LOB', 'U') IS NOT NULL
    DROP TABLE dbo.REF_LOB;
GO

CREATE TABLE dbo.REF_LOB (

```

Schema Layer

```
LOB_CD VARCHAR(10) PRIMARY KEY,  
LOB_DESC VARCHAR(50) NOT NULL  
);  
GO  
  
-- Populating with valid LOB mappings  
INSERT INTO dbo.REF_LOB (LOB_CD, LOB_DESC)  
VALUES  
    ('AUTO', 'Automobile Insurance'),  
    ('HOME', 'Homeowners Insurance'),  
    ('RENTER', 'Renters Insurance');  
GO  
  
*/  
  
-- SELECT * FROM dbo.REF_STATE;
```



The screenshot shows the SQL Server Enterprise Manager interface. The query editor at the top contains the following SQL code:

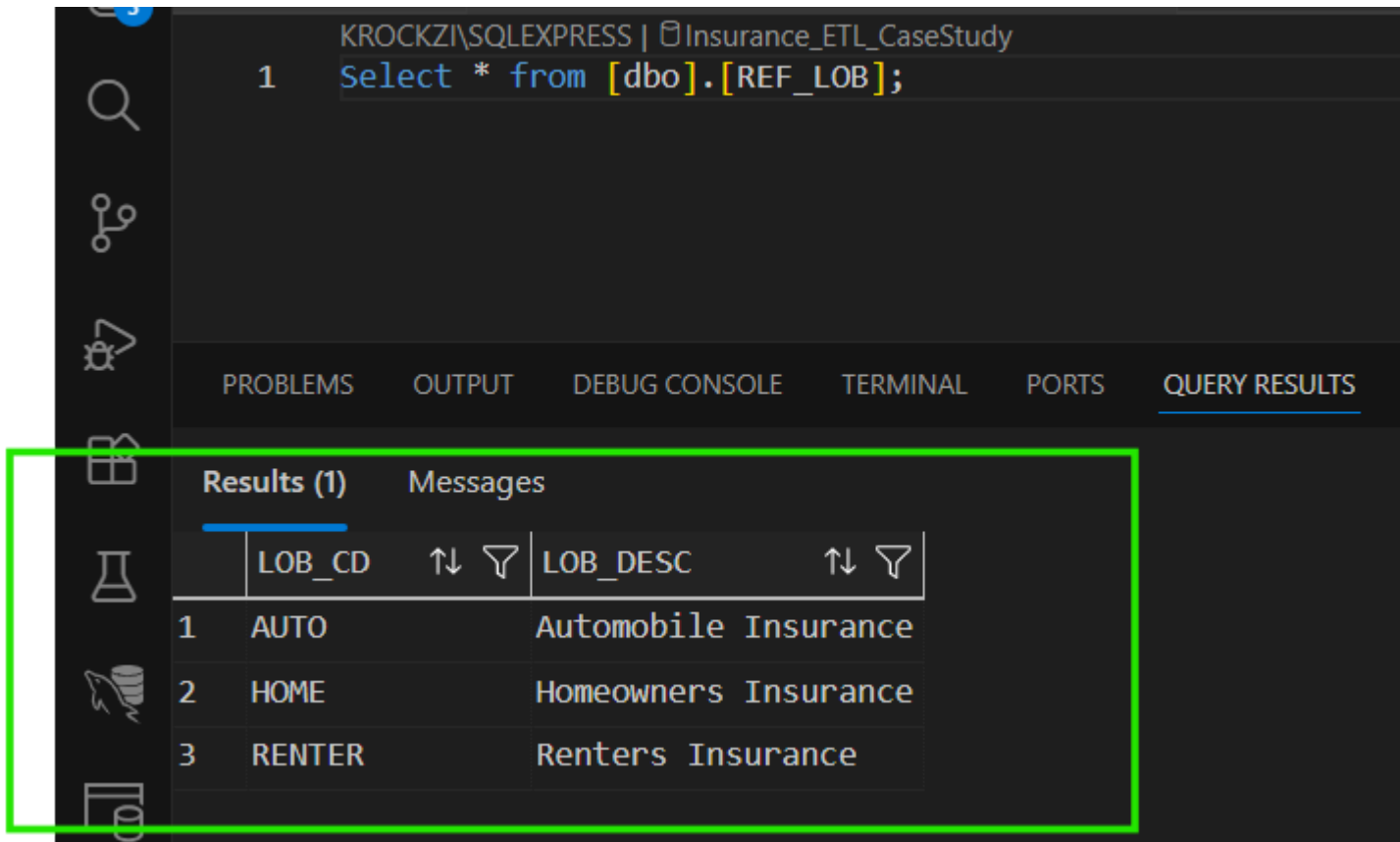
```
1  Select * from [dbo].[REF_STATE];
```

The interface includes tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, PORTS, QUERY RESULTS, and AZURE. The QUERY RESULTS tab is active, displaying the results of the query. The results are shown in a table with the following data:

	STATE_CD
1	CA
2	FL
3	IL
4	NY
5	TX

```
-- SELECT * FROM  
-- dbo.REF_LOB;
```

Schema Layer



The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a query: `select * from [dbo].[REF_LOB];`. The bottom pane shows the query results under the 'QUERY RESULTS' tab. The results are displayed in a table with two columns: 'LOB_CD' and 'LOB_DESC'. The table contains three rows of data: 1. AUTO, Automobile Insurance; 2. HOME, Homeowners Insurance; 3. RENTER, Renters Insurance. The results pane is highlighted with a green rectangle.

	LOB_CD	LOB_DESC
1	AUTO	Automobile Insurance
2	HOME	Homeowners Insurance
3	RENTER	Renters Insurance

-- Target Table

/*

-- Drop table if it already exists

IF OBJECT_ID('dbo.ILM_POLICY', 'U') IS NOT NULL

DROP TABLE dbo.ILM_POLICY;

GO

CREATE TABLE dbo.ILM_POLICY (

-- Cleaned & Transformed Columns

POLICY_NUMBER VARCHAR(20) NOT NULL,

EFFECTIVE_DATE DATE NULL, -- Strict DATE type

PREMIUM_AMOUNT DECIMAL(14,2) NOT NULL, -- Increased precision per STTM

STATE_CODE CHAR(2) NULL,

LINE_OF_BUSINESS VARCHAR(50) NULL,

INSURED_NAME VARCHAR(100) NULL,

STATUS VARCHAR(10) NULL,

-- Audit Metadata (System timestamp at load)

LOAD_TS DATETIME2 DEFAULT SYSDATETIME() NOT NULL

);

GO

*/

-- Error Table

-- Drop table if it already exists

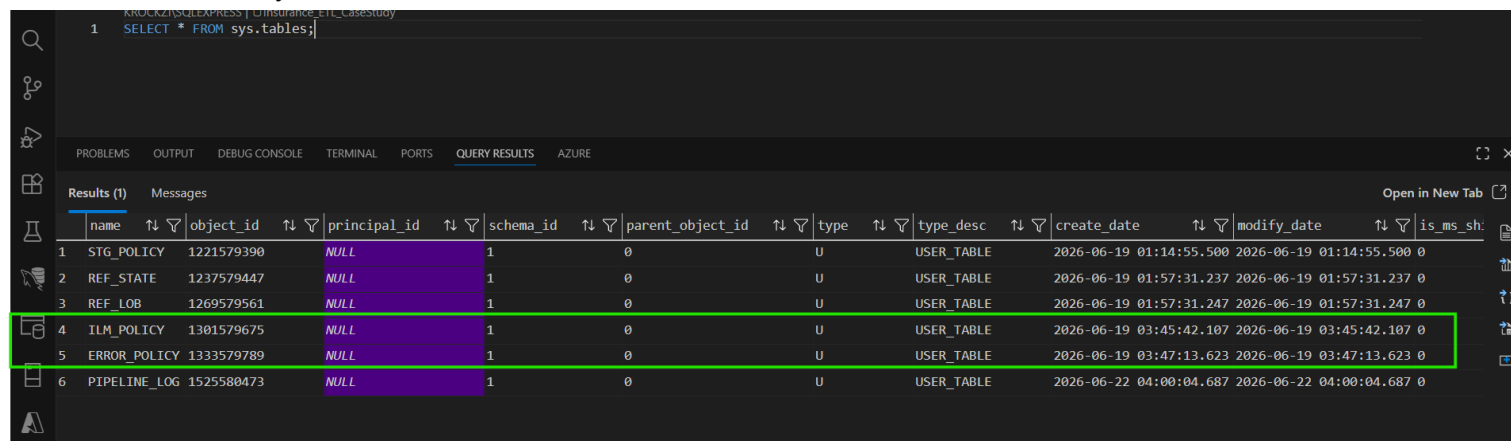
Schema Layer

```
/*IF OBJECT_ID('dbo.ERROR_POLICY', 'U') IS NOT NULL
    DROP TABLE dbo.ERROR_POLICY;
GO
```

```
CREATE TABLE dbo.ERROR_POLICY (
    -- Raw Staging Columns (Keeping them as strings/raw formats to prevent crashes)
    POL_NBR          VARCHAR(20)      NULL,
    EFF_DT           VARCHAR(8) NULL,
    PREM_AMT         DECIMAL(12,2) NULL,
    ST_CD            CHAR(2)      NULL,
    LOB_CD           VARCHAR(10)      NULL,
    LNAME            VARCHAR(40)      NULL,
    FNAME            VARCHAR(40)      NULL,
    CANCEL_FLG       CHAR(1)         NULL,
    SRC_UPDATE_TS    DATETIME2       NULL,

    -- Error Metadata (The "Documented Reject" columns)
    REJECT_REASON    VARCHAR(255) NOT NULL,
    REJECT_TS        DATETIME2       DEFAULT SYSDATETIME() NOT NULL
);
GO
*/
```

```
--SELECT * FROM sys.tables;
```



1 SELECT * FROM sys.tables;

	name	object_id	principal_id	schema_id	parent_object_id	type	type_desc	create_date	modify_date	is_ms_sh:
1	STG_POLICY	1221579390	NULL	1	0	U	USER_TABLE	2026-06-19 01:14:55.500	2026-06-19 01:14:55.500	0
2	REF_STATE	1237579447	NULL	1	0	U	USER_TABLE	2026-06-19 01:57:31.237	2026-06-19 01:57:31.237	0
3	REF_LOB	1269579561	NULL	1	0	U	USER_TABLE	2026-06-19 01:57:31.247	2026-06-19 01:57:31.247	0
4	ILM_POLICY	1301579675	NULL	1	0	U	USER_TABLE	2026-06-19 03:45:42.107	2026-06-19 03:45:42.107	0
5	ERROR_POLICY	1333579789	NULL	1	0	U	USER_TABLE	2026-06-19 03:47:13.623	2026-06-19 03:47:13.623	0
6	PIPELINE_LOG	1525580473	NULL	1	0	U	USER_TABLE	2026-06-22 04:00:04.687	2026-06-22 04:00:04.687	0

Transformation Layer

Transformation Layer

```
/*-- =====
-- 1. CLEAN SLATE & ENVIRONMENT CLEANUP (Idempotency Checks)
-- =====
-- Wipe out historical test runs to clear old 1900-01-01 data bugs
TRUNCATE TABLE dbo.ILM_POLICY;
TRUNCATE TABLE dbo.ERROR_POLICY;

-- Drop temp processing blocks if they exist from a crashed run
IF OBJECT_ID('tempdb..#TempRoutedData') IS NOT NULL
    DROP TABLE #TempRoutedData;
GO

-- =====
-- 2. THE MASTER TRANSFORMATION PIPELINE
-- =====

WITH
-- Station 1: The Deduplication Layer (Isolate the valid grain)
cte_dedup AS (
    SELECT
        POL_NBR, EFF_DT, PREM_AMT, ST_CD, LOB_CD, LNAME, FNAME, CANCEL_FLG, SRC_UPDATE_TS,
        ROW_NUMBER() OVER(
            PARTITION BY POL_NBR
            ORDER BY
                SRC_UPDATE_TS DESC,
                COALESCE(PREM_AMT, 0.00) DESC -- Tie-Breaker for exact millisecond collisions
        ) as RowRank
    FROM dbo.STG_POLICY
),

-- Station 2 & 3: Cleansing and Reference Lookups
cte_transform AS (
```

Transformation Layer

```
SELECT
-- A. PRESERVE RAW COLUMNS (Needed for Error Table if the row fails)
d.POL_NBR      AS RAW_POL_NBR,
d.EFF_DT       AS RAW_EFF_DT,
d.PREM_AMT     AS RAW_PREM_AMT,
d.ST_CD        AS RAW_ST_CD,
d.LOB_CD       AS RAW_LOB_CD,
d.LNAME        AS RAW_LNAME,
d.FNAME        AS RAW_FNAME,
d.CANCEL_FLG   AS RAW_CANCEL_FLG,
d.SRC_UPDATE_TS AS RAW_SRC_UPDATE_TS,

-- B. TRANSFORM & VALIDATE TARGET COLUMNS (For ILM_POLICY)
TRIM(d.POL_NBR) AS TGT_POL_NBR,
TRY_CONVERT DATE, NULLIF(TRIM(d.EFF_DT), ''), 112 AS TGT_EFF_DT,

-- Default Premium to 0.00 safely
CAST(COALESCE(d.PREM_AMT, 0.00) AS DECIMAL(14,2)) AS TGT_PREM_AMT,

-- State Lookup (Upper/Trim to clean string before matching)
rs.STATE_CD AS TGT_ST_CD,

-- LOB Lookup (Default to 'UNKNOWN' if no match)
COALESCE(r.l.LOB_DESC, 'UNKNOWN') AS TGT_LOB,

-- Safe Concatenation (CONCAT_WS ignores NULLs to prevent data wipeout)
CONCAT_WS(' ', NULLIF(TRIM(d.LNAME), ''), NULLIF(TRIM(d.FNAME), '')) AS TGT_NAME,

-- Decode Status Flag
CASE UPPER(TRIM(d.CANCEL_FLG))
WHEN 'Y' THEN 'CANCELLED'
```

Transformation Layer

```
        WHEN 'N' THEN 'ACTIVE'
        ELSE 'UNKNOWN'
    END AS TGT_STATUS

FROM cte_dedup d
LEFT JOIN dbo.REF_STATE rs ON UPPER(TRIM(d.ST_CD)) = rs.STATE_CD
LEFT JOIN dbo.REF_LOB rl  ON TRIM(d.LOB_CD) = rl.LOB_CD
WHERE d.RowRank = 1
),

-- Station 4: The Routing Engine (The Inspector)
cte_routing AS (
    SELECT
        *,
        CASE
            -- STRICT DATE ROUTING: Catches garbage text, blanks ("), and NULLs cleanly
            WHEN TGT_EFF_DT IS NULL
                THEN 'INVALID_OR_MISSING_DATE'

            -- Trap invalid or missing states
            WHEN TGT_ST_CD IS NULL
                THEN 'INVALID_OR_MISSING_STATE_CODE'

            -- Everything passed successfully
            ELSE 'VALID'
        END AS ROUTING_FLAG
    FROM cte_transform
)

-- Execute the CTE pipeline and dump results into a temporary table in RAM
SELECT * INTO #TempRoutedData
```

Transformation Layer

FROM cte_routing;

GO

-- =====

-- 3. THE DATA SPLIT (Load the physical tables)

-- =====

-- A. INCREMENTAL UPSERT: Merge clean data into Target Table (ILM_POLICY)

MERGE dbo.ILM_POLICY AS TGT

USING (

SELECT

TGT_POL_NBR, TGT_EFF_DT, TGT_PREM_AMT, TGT_ST_CD,

TGT_LOB, TGT_NAME, TGT_STATUS

FROM #TempRoutedData

WHERE ROUTING_FLAG = 'VALID'

) AS SRC

ON TGT.POLICY_NUMBER = SRC.TGT_POL_NBR

-- If the policy number already matches, update the existing system record

WHEN MATCHED THEN

UPDATE SET

TGT.EFFECTIVE_DATE = SRC.TGT_EFF_DT,

TGT.PREMIUM_AMOUNT = SRC.TGT_PREM_AMT,

TGT.STATE_CODE = SRC.TGT_ST_CD,

TGT.LINE_OF_BUSINESS = SRC.TGT_LOB,

TGT.INSURED_NAME = SRC.TGT_NAME,

TGT.STATUS = SRC.TGT_STATUS -- Captures soft-delete status flips

-- If it's a completely fresh policy record, insert it

WHEN NOT MATCHED THEN

INSERT (

Transformation Layer

```
POLICY_NUMBER, EFFECTIVE_DATE, PREMIUM_AMOUNT, STATE_CODE,
LINE_OF_BUSINESS, INSURED_NAME, STATUS
)
VALUES (
SRC.TGT_POL_NBR, SRC.TGT_EFF_DT, SRC.TGT_PREM_AMT, SRC.TGT_ST_CD,
SRC.TGT_LOB, SRC.TGT_NAME, SRC.TGT_STATUS
);

-- B. AUDIT TRAIL INSERT: Load the Error Quarantine Table (ERROR_POLICY)
INSERT INTO dbo.ERROR_POLICY (
POL_NBR, EFF_DT, PREM_AMT, ST_CD, LOB_CD, LNAME, FNAME,
CANCEL_FLG, SRC_UPDATE_TS, REJECT_REASON
)
SELECT
RAW_POL_NBR, RAW_EFF_DT, RAW_PREM_AMT, RAW_ST_CD, RAW_LOB_CD,
RAW_LNAME, RAW_FNAME, RAW_CANCEL_FLG, RAW_SRC_UPDATE_TS,
ROUTING_FLAG
FROM #TempRoutedData
WHERE ROUTING_FLAG != 'VALID';
GO

-- =====
-- 4. CLEANUP
-- =====

DROP TABLE #TempRoutedData;
GO */
```

Transformation Layer

– SELECT * FROM [dbo].[ILM_POLICY];

KROCKZI\SQLEXPRESS | Insurance_ETL_CaseStudy

1 SELECT * FROM [dbo].[ILM_POLICY];

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE

Results (1) Messages

Open in New Tab

	POLICY_NUMBER	EFFECTIVE_DATE	PREMIUM_AMOUNT	STATE_CODE	LINE_OF_BUSINESS	INSURED_NAME	STATUS	LOAD_TS
1	POL-80001	2023-12-01	100.00	FL	Automobile Insurance	Clark, Kenneth	CANCELLED	2026-06-28 15:16:53.3949534
2	POL-10001	2023-10-01	1500.00	TX	Automobile Insurance	Smith, John	ACTIVE	2026-06-28 15:16:53.3949534
3	POL-10002	2023-10-02	850.00	CA	Homeowners Insurance	Johnson, Mary	ACTIVE	2026-06-28 15:16:53.3949534
4	POL-10003	2023-10-03	400.25	NY	Renters Insurance	Williams, James	ACTIVE	2026-06-28 15:16:53.3949534
5	POL-10004	2023-10-04	2100.00	FL	Automobile Insurance	Brown, Patricia	CANCELLED	2026-06-28 15:16:53.3949534
6	POL-10005	2023-10-05	999.99	IL	Homeowners Insurance	Jones, Robert	ACTIVE	2026-06-28 15:16:53.3949534
7	POL-10006	2023-10-06	350.00	TX	Renters Insurance	Garcia, Linda	CANCELLED	2026-06-28 15:16:53.3949534
8	POL-10007	2023-10-07	1750.75	CA	Automobile Insurance	Miller, Michael	ACTIVE	2026-06-28 15:16:53.3949534
9	POL-10008	2023-10-08	1100.00	NY	Homeowners Insurance	Davis, Barbara	ACTIVE	2026-06-28 15:16:53.3949534
10	POL-10009	2023-10-09	200.00	FL	Renters Insurance	Rodriguez, William	ACTIVE	2026-06-28 15:16:53.3949534
11	POL-10010	2023-10-10	3200.00	IL	Automobile Insurance	Martinez, Richard	CANCELLED	2026-06-28 15:16:53.3949534
12	POL-11001	2024-02-01	1050.00	TX	Automobile Insurance	Adams, John	ACTIVE	2026-06-28 15:16:53.3949534
13	POL-11002	2024-02-02	1200.25	NY	Homeowners Insurance	Baker, Sarah	ACTIVE	2026-06-28 15:16:53.3949534
14	POL-11003	2024-02-03	450.00	FL	Renters Insurance	Clark, David	ACTIVE	2026-06-28 15:16:53.3949534
15	POL-11004	2024-02-04	2100.50	IL	Automobile Insurance	Davis, Emily	ACTIVE	2026-06-28 15:16:53.3949534
16	POL-11005	2024-02-05	1500.00	CA	Homeowners Insurance	Evans, Michael	ACTIVE	2026-06-28 15:16:53.3949534
17	POL-11006	2024-02-06	300.00	TX	Renters Insurance	Ford, Jessica	ACTIVE	2026-06-28 15:16:53.3949534
18	POL-11007	2024-02-07	1340.75	NY	Automobile Insurance	Garcia, Robert	ACTIVE	2026-06-28 15:16:53.3949534

Ln 1, Col 34 Spaces: 4 UTF-8 CRLF () SQL MSSQL SQLCMD: Off 50 rows affected 00:00:00.021 KROCKZI\SQLEXPRESS Insurance_ETL_CaseStudy

– SELECT * FROM [dbo].[ERROR_POLICY];

KROCKZI\SQLEXPRESS | Insurance_ETL_CaseStudy

1 SELECT * FROM [dbo].[ERROR_POLICY];

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE

Results (1) Messages

Open in New Tab

Result Set Batch 1 - Query 1

=====

POL_NBR	EFF_DT	PREM_AMT	ST_CD	LOB_CD	LNAME	FNAME	CANCEL_FLG	SRC_UPDATE_TS	REJECT_REASON	REJECT_TS
POL-30001	20230229	500.00	NY	AUTO	Gonzalez	Thomas	N	2023-11-05 08:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-30002	20231301	450.00	FL	HOME	Wilson	Sarah	N	2023-11-05 09:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-30003	ABCDEF GH	300.00	TX	RENTER	Anderson	Matthew	N	2023-11-05 10:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-30004		700.00	CA	AUTO	Thomas	Karen	N	2023-11-05 11:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-30005	NULL	800.00	IL	HOME	Taylor	Nancy	N	2023-11-05 12:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-40002	20231110	1300.00	ZZ	HOME	Jackson	Betty	N	2023-11-10 09:00:00.0000000	INVALID_OR_MISSING_STATE_CODE	2026-06-28 15:16:53.3968582
POL-40003	20231110	1400.00	NULL	RENTER	Martin	Margaret	N	2023-11-10 10:00:00.0000000	INVALID_OR_MISSING_STATE_CODE	2026-06-28 15:16:53.3968582
POL-40004	20231110	1500.00	C	AUTO	Lee	Sandra	N	2023-11-10 11:00:00.0000000	INVALID_OR_MISSING_STATE_CODE	2026-06-28 15:16:53.3968582
POL-80003	20240515	850.00	NULL	AUTO	Doe	Jane	N	2026-06-22 01:46:11.6166667	INVALID_OR_MISSING_STATE_CODE	2026-06-28 15:16:53.3968582

(9 rows affected)

Reconciliation

Reconciliation

```
WITH Missing_Data_Check AS (  
    SELECT DISTINCT TRIM(POL_NBR) AS Missing_Policies  
    FROM dbo.STG_POLICY  
    EXCEPT  
    (  
        SELECT TRIM(POLICY_NUMBER) FROM dbo.ILM_POLICY  
        UNION  
        SELECT TRIM(POL_NBR) FROM dbo.ERROR_POLICY  
    )  
)  
  
-- SCENARIO A: The query found missing data  
  
SELECT  
    'WARNING: Missing Record Detected' AS Alert_Message,  
    'Investigate source data immediately' AS Recommended_Action,  
    Missing_Policies  
FROM Missing_Data_Check  
  
UNION ALL  
  
-- SCENARIO B: The query found absolutely nothing  
  
SELECT  
    'SUCCESS: Zero Drops' AS Alert_Message,  
    'Pipeline balanced perfectly' AS Recommended_Action,  
    'NONE' AS Missing_Policies  
  
WHERE NOT EXISTS (SELECT 1 FROM Missing_Data_Check);  
  
GO
```

Reconciliation

```
KROCKZI\SQLEXPRESS | Insurance_ETL_CaseStudy
1  WITH Missing_Data_Check AS (
2      SELECT DISTINCT TRIM(POL_NBR) AS Missing_Policies
3      FROM dbo.STG_POLICY
4      EXCEPT
5      (
6          SELECT TRIM(POLICY_NUMBER) FROM dbo.ILM_POLICY
7          UNION
8          SELECT TRIM(POL_NBR) FROM dbo.ERROR_POLICY
9      )
10 )
11 -- SCENARIO A: The query found missing data
12 SELECT
13     'WARNING: Missing Record Detected' AS Alert_Message,
14     'Investigate source data immediately' AS Recommended_Action,
15     Missing_Policies
16 FROM Missing_Data_Check
17 UNION ALL
18 -- SCENARIO B: The query found absolutely nothing
19 SELECT
20     'SUCCESS: Zero Drops' AS Alert_Message,
21     'Pipeline balanced perfectly' AS Recommended_Action,
22     'NONE' AS Missing_Policies
23 WHERE NOT EXISTS (SELECT 1 FROM Missing_Data_Check);
24 GO
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

QUERY RESULTS

AZURE

Results (1)

Messages

	Alert_Message	Recommended_Action	Missing_Policies
1	SUCCESS: Zero Drops	Pipeline balanced perfectly	NONE

Data Load Scenarios

Data Load Scenarios

Scenario: Incremental Load & Automated Error Routing

Objective: Process a new daily batch of staging data containing existing updates, net-new records, and invalid data without duplicating records or breaking the pipeline.

New data inserted in source table:

```
/* INSERT INTO dbo.STG_POLICY (
    POL_NBR, EFF_DT, PREM_AMT, ST_CD, LOB_CD, LNAME, FNAME, CANCEL_FLG, SRC_UPDATE_TS
)
VALUES
    -- Row 1: Valid Data
    ('POL-80002', '20240501', 1250.50, 'NY', 'HOME', 'Smith', 'John', 'N', GETDATE()),

    -- Row 2: Invalid Data
    ('POL-80003', '20240515', 850.00, NULL, 'AUTO', 'Doe', 'Jane', 'N', GETDATE());
*/
```

Post Transformation -

Net-New Records (Inserts): 1-new, valid policy (POL-80002) was identified by the MERGE statement and cleanly appended to dbo.ILM_POLICY.

Target Table :

KROCKZI\SQLEXPRESS | Insurance_ETL_CaseStudy

1 Select * from [dbo].[ILM_POLICY];

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE

Results (1) Messages

	POLICY_NUMBER	EFFECTIVE_DATE	PREMIUM_AMOUNT	STATE_CODE	LINE_OF_BUSINESS	INSURED_NAME	STATUS	LOAD_TS
3	POL-60002	2023-11-20	0.00	FL	Homeowners Insurance	Harris, Andrew	ACTIVE	2026-06-28 15:16:53.3949534
4	POL-70001	2023-11-25	400.00	TX	Renters Insurance	Joshua	ACTIVE	2026-06-28 15:16:53.3949534
5	POL-70002	2023-11-25	450.00	CA	Automobile Insurance	Sanchez	ACTIVE	2026-06-28 15:16:53.3949534
6	POL-70003	2023-11-25	500.00	NY	Homeowners Insurance		ACTIVE	2026-06-28 15:16:53.3949534
7	POL-80002	2024-05-01	1250.50	NY	Homeowners Insurance	Smith, John	ACTIVE	2026-06-28 15:16:53.3949534
8	POL-80004	2023-12-01	250.00	NY	Automobile Insurance	Robinson, George	UNKNOWN	2026-06-28 15:16:53.3949534
9	POL-90001	2023-12-10	750.00	FL	Homeowners Insurance	Walker, Edward	ACTIVE	2026-06-28 15:16:53.3949534

Open in New

Data Load Scenarios

Data Quality Quarantine: 1 invalid policy (POL-80003 with a NULL State Code) was intercepted by the routing engine, bypassed the target table entirely, and was successfully quarantined in dbo.ERROR_POLICY.

Error Table :

KROCKZ\SQLSERVER | U\Insurance_ETL_CaseStudy

1 -- Select * from [dbo].[ILM_POLICY];

2 Select * from [dbo].[ERROR_POLICY];

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE

Results (1) Messages

Open in New Tab

Result Set Batch 1 - Query 1

POL_NBR	EFF_DT	PREM_AMT	ST_CD	LOB_CD	LNAME	FNAME	CANCEL_FLG	SRC_UPDATE_TS	REJECT_REASON	REJECT_TS
POL-30001	20230229	500.00	NY	AUTO	Gonzalez	Thomas	N	2023-11-05 08:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-30002	20231301	450.00	FL	HOME	Wilson	Sarah	N	2023-11-05 09:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-30003	ABCDEFGHI	300.00	TX	RENTER	Anderson	Matthew	N	2023-11-05 10:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-30004		700.00	CA	AUTO	Thomas	Karen	N	2023-11-05 11:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-30005	NULL	800.00	IL	HOME	Taylor	Nancy	N	2023-11-05 12:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-28 15:16:53.3968582
POL-40002	20231110	1300.00	ZZ	HOME	Jackson	Betty	N	2023-11-10 09:00:00.0000000	INVALID_OR_MISSING_STATE_CODE	2026-06-28 15:16:53.3968582
POL-40003	20231110	1400.00	NULL	RENTER	Martin	Margaret	N	2023-11-10 10:00:00.0000000	INVALID_OR_MISSING_STATE_CODE	2026-06-28 15:16:53.3968582
POL-40004	20231110	1500.00	C	AUTO	Lee	Sandra	N	2023-11-10 11:00:00.0000000	INVALID_OR_MISSING_STATE_CODE	2026-06-28 15:16:53.3968582
POL-80003	20240515	850.00	NULL	AUTO	Doe	Jane	N	2026-06-22 01:46:11.6166667	INVALID_OR_MISSING_STATE_CODE	2026-06-28 15:16:53.3968582

(9 rows affected)

Scenario 2: The True Update (SCD Type 1 Overwrite)

Objective: Process an incoming record for an existing policy where the business data (e.g., Premium Amount) has changed, ensuring the target database reflects the most current state of the real world.

Modified Record: 1 existing policy (POL-10001) was successfully identified by the pipeline. The old premium of \$1,200.50 was permanently overwritten in place with the updated \$1,500.00 premium.

```
/*UPDATE dbo.STG_POLICY
SET
    PREM_AMT = 1500.00,
    SRC_UPDATE_TS = CURRENT_TIMESTAMP
WHERE POL_NBR = 'POL-10001';
*/
```

Target Table :

Post Transformation –

Data Load Scenarios

1SELECT * FROM [dbo].[ILM_POLICY];

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

QUERY RESULTS

AZURE

Results (1)

Messages

Open in New

POLICY_NUMBER	EFFECTIVE_DATE	PREMIUM_AMOUNT	STATE_CODE	LINE_OF_BUSINESS	INSURED_NAME	STATUS	LOAD_TS
POL-80001	2023-12-01	100.00	FL	Automobile Insurance	Clark, Kenneth	CANCELLED	2026-06-28 15:16:53.3949534
POL-10001	2023-10-01	1500.00	TX	Automobile Insurance	Smith, John	ACTIVE	2026-06-28 15:16:53.3949534
POL-10002	2023-10-02	850.00	CA	Homeowners Insurance	Johnson, Mary	ACTIVE	2026-06-28 15:16:53.3949534

Scenario 3: The Exact Duplicate (Compute Optimization)

Objective: Prevent the pipeline from processing redundant updates when incoming staging data is an edge-to-edge perfect match with the existing target data.

Pipeline Execution Results:

- Bypassed Records:** Incoming policies that perfectly matched the existing records in the target table were identified and cleanly ignored.
- Zero Write Operations:** The optimized pipeline executed with (0 rows affected) for these specific rows, eliminating unnecessary database I/O, compute waste, and index fragmentation.

Change needed in existing merge query:

```
MERGE INTO dbo.ILM_POLICY AS Target
USING (
    SELECT * FROM #TempRoutedData WHERE ROUTING_FLAG = 'VALID'
) AS Source
ON Target.POLICY_NUMBER = Source.TGT_POL_NBR
WHEN MATCHED AND (
    Target.EFFECTIVE_DATE <> Source.TGT_EFF_DT OR
    Target.PREMIUM_AMOUNT <> Source.TGT_PREM_AMT OR
    Target.STATE_CODE <> Source.TGT_ST_CD OR
    Target.LINE_OF_BUSINESS <> Source.TGT_LOB OR
    Target.INSURED_NAME <> Source.TGT_NAME OR
    Target.STATUS <> Source.TGT_STATUS
)
THEN
    UPDATE SET
        Target.EFFECTIVE_DATE = Source.TGT_EFF_DT,
        Target.PREMIUM_AMOUNT = Source.TGT_PREM_AMT,
        Target.STATE_CODE = Source.TGT_ST_CD,
```

Data Load Scenarios

```
Target.LINE_OF_BUSINESS = Source.TGT_LOB,
```

```
Target.INSURED_NAME      = Source.TGT_NAME,
```

```
Target.STATUS            = Source.TGT_STATUS,
```

```
Target.LOAD_TS           = GETDATE()
```

```
WHEN NOT MATCHED BY TARGET THEN
```

```
    INSERT (
```

```
        POLICY_NUMBER, EFFECTIVE_DATE, PREMIUM_AMOUNT, STATE_CODE,
```

```
        LINE_OF_BUSINESS, INSURED_NAME, STATUS, LOAD_TS
```

```
    )
```

```
    VALUES (
```

```
        Source.TGT_POL_NBR, Source.TGT_EFF_DT, Source.TGT_PREM_AMT, Source.TGT_ST_CD,
```

```
        Source.TGT_LOB, Source.TGT_NAME, Source.TGT_STATUS, GETDATE()
```

```
    );
```

```
INSERT INTO dbo.ERROR_POLICY (
```

```
    POL_NBR, EFF_DT, PREM_AMT, ST_CD, LOB_CD, LNAME, FNAME,
```

```
    CANCEL_FLG, SRC_UPDATE_TS, REJECT_REASON
```

```
)
```

```
SELECT
```

```
    RAW_POL_NBR, RAW_EFF_DT, RAW_PREM_AMT, RAW_ST_CD, RAW_LOB_CD,
```

```
    RAW_LNAME, RAW_FNAME, RAW_CANCEL_FLG, RAW_SRC_UPDATE_TS,
```

```
    ROUTING_FLAG
```

```
FROM #TempRoutedData
```

```
WHERE ROUTING_FLAG != 'VALID';
```

```
GO
```

Note: -- Simulate a bulk batch of 15 net-new policies arriving in the staging file

```
INSERT INTO dbo.STG_POLICY (
```

```
    POL_NBR, EFF_DT, PREM_AMT, ST_CD, LOB_CD, LNAME, FNAME, CANCEL_FLG, SRC_UPDATE_TS
```

```
)
```

```
VALUES
```

```
    ('POL-11001', '20240201', 1050.00, 'TX', 'AUTO', 'Adams', 'John', 'N', GETDATE()),
```

```
    ('POL-11002', '20240202', 1200.25, 'NY', 'HOME', 'Baker', 'Sarah', 'N', GETDATE()),
```

```
    ('POL-11003', '20240203', 450.00, 'FL', 'RENTER', 'Clark', 'David', 'N', GETDATE()),
```

```
    ('POL-11004', '20240204', 2100.50, 'IL', 'AUTO', 'Davis', 'Emily', 'N', GETDATE()),
```

```
    ('POL-11005', '20240205', 1500.00, 'CA', 'HOME', 'Evans', 'Michael', 'N', GETDATE()),
```

```
    ('POL-11006', '20240206', 300.00, 'TX', 'RENTER', 'Ford', 'Jessica', 'N', GETDATE()),
```

```
    ('POL-11007', '20240207', 1340.75, 'NY', 'AUTO', 'Garcia', 'Robert', 'N', GETDATE()),
```

```
    ('POL-11008', '20240208', 2200.00, 'FL', 'HOME', 'Hall', 'Ashley', 'N', GETDATE()),
```

```
    ('POL-11009', '20240209', 350.25, 'IL', 'RENTER', 'Irwin', 'James', 'N', GETDATE()),
```

```
    ('POL-11010', '20240210', 1750.00, 'CA', 'AUTO', 'Jones', 'Mary', 'N', GETDATE()),
```

```
    ('POL-11011', '20240211', 1100.00, 'TX', 'HOME', 'King', 'William', 'N', GETDATE()),
```

Data Load Scenarios

```
('POL-11012', '20240212', 410.50, 'NY', 'RENTER', 'Lee', 'Patricia', 'N', GETDATE()),  
( 'POL-11013', '20240213', 1980.00, 'FL', 'AUTO', 'Moore', 'Richard', 'N', GETDATE()),  
( 'POL-11014', '20240214', 1650.25, 'IL', 'HOME', 'Nelson', 'Linda', 'N', GETDATE()),  
( 'POL-11015', '20240215', 290.00, 'CA', 'RENTER', 'Ortiz', 'Charles', 'N', GETDATE());
```

*/

Validation

1. Schema / Metadata Validation

Validated the target table schema to ensure the column names, data types, lengths, and precision are aligned with the Source-to-Target Mapping (STTM).

```
SELECT
    COLUMN_NAME,
    DATA_TYPE,
    CHARACTER_MAXIMUM_LENGTH,
    NUMERIC_PRECISION,
    NUMERIC_SCALE,
    IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo'
    AND TABLE_NAME = 'ILM_POLICY'
ORDER BY ORDINAL_POSITION;
```

1

2

3

4

5

6

7

8

9

10

11

```
SELECT
    COLUMN_NAME,
    DATA_TYPE,
    CHARACTER_MAXIMUM_LENGTH,
    NUMERIC_PRECISION,
    NUMERIC_SCALE,
    IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo'
    AND TABLE_NAME = 'ILM_POLICY'
ORDER BY ORDINAL_POSITION;
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

QUERY RESULTS

AZURE

Results (1)

Messages

Result Set Batch 1 - Query 1

=====

COLUMN_NAME	DATA_TYPE	CHARACTER_MAXIMUM_LENGTH	NUMERIC_PRECISION	NUMERIC_SCALE	IS_NULLABLE
POLICY_NUMBER	varchar	20	NULL	NULL	NO
EFFECTIVE_DATE	date	NULL	NULL	NULL	YES
PREMIUM_AMOUNT	decimal	NULL	14	2	NO
STATE_CODE	char	2	NULL	NULL	YES
LINE_OF_BUSINESS	varchar	50	NULL	NULL	YES
INSURED_NAME	varchar	100	NULL	NULL	YES
STATUS	varchar	10	NULL	NULL	YES
LOAD_TS	datetime2	NULL	NULL	NULL	NO

(8 rows affected)

Validation

Hash-Based Validation

Performed hash-based validation by recomputing the expected transformed dataset from the source using the STTM business rules. A SHA2-256 hash was generated for each expected row and compared with the corresponding hash in the target table. Matching hashes confirm the target data is consistent with the expected transformation, while mismatches identify missing, extra, or incorrectly transformed records.

WITH cte_dedup AS

```
(
    SELECT *,
    ROW_NUMBER() OVER
    (
        PARTITION BY POL_NBR
        ORDER BY SRC_UPDATE_TS DESC,
        COALESCE(PREM_AMT,0) DESC
    ) rn
    FROM dbo.STG_POLICY
),
```

Expected AS

```
(
    SELECT
    TRIM(d.POL_NBR) AS POLICY_NUMBER,
    TRY_CONVERT(DATE,NULLIF(TRIM(d.EFF_DT),"),112) AS EFFECTIVE_DATE,
    CAST(COALESCE(d.PREM_AMT,0.00) AS DECIMAL(14,2)) AS PREMIUM_AMOUNT,
    rs.STATE_CD AS STATE_CODE,
    COALESCE(rl.LOB_DESC,'UNKNOWN') AS LINE_OF_BUSINESS,
    CONCAT_WS(' ',
        NULLIF(TRIM(d.LNAME),"),
        NULLIF(TRIM(d.FNAME),")
    ) AS INSURED_NAME,
    CASE UPPER(TRIM(d.CANCEL_FLG))
    WHEN 'Y' THEN 'CANCELLED'
    WHEN 'N' THEN 'ACTIVE'
```

Validation

```
ELSE 'UNKNOWN'
END AS STATUS
FROM cte_dedup d
LEFT JOIN dbo.REF_STATE rs
ON UPPER(TRIM(d.ST_CD)) = rs.STATE_CD
LEFT JOIN dbo.REF_LOB rl
ON TRIM(d.LOB_CD) = rl.LOB_CD
WHERE rn = 1
AND TRY_CONVERT(DATE,NULLIF(TRIM(d.EFF_DT), ''),112) IS NOT NULL
AND rs.STATE_CD IS NOT NULL
),
```

ExpectedHash AS

```
(
    SELECT
        POLICY_NUMBER,
        HASHBYTES
        (
            'SHA2_256',
            CONCAT(
                POLICY_NUMBER,'|',
                CONVERT(VARCHAR(10),EFFECTIVE_DATE,120),'|',
                PREMIUM_AMOUNT,'|',
                STATE_CODE,'|',
                LINE_OF_BUSINESS,'|',
                INSURED_NAME,'|',
                STATUS
            )
        ) AS RowHash
    FROM Expected
),
```

Validation

TargetHash AS

```
(  
    SELECT  
        POLICY_NUMBER,  
        HASHBYTES  
        (  
            'SHA2_256',  
            CONCAT(  
                POLICY_NUMBER,'|',  
                CONVERT(VARCHAR(10),EFFECTIVE_DATE,120),'|',  
                PREMIUM_AMOUNT,'|',  
                STATE_CODE,'|',  
                LINE_OF_BUSINESS,'|',  
                INSURED_NAME,'|',  
                STATUS  
            )  
        ) AS RowHash  
    FROM dbo.ILM_POLICY  
)
```

SELECT *

INTO #HashValidation

FROM

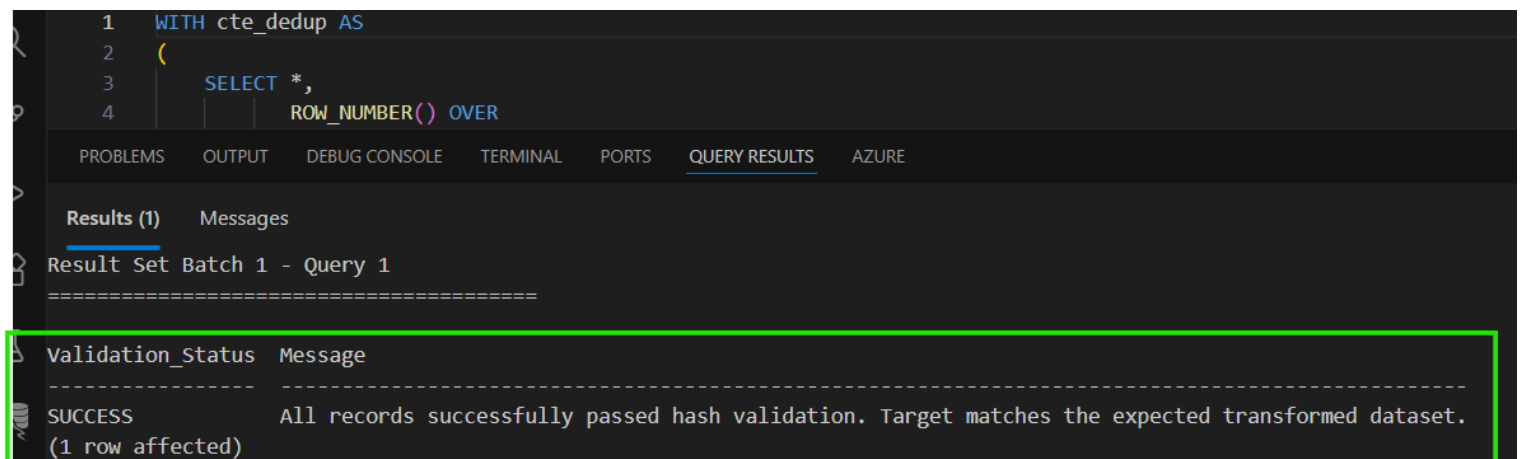
```
(  
    SELECT  
        COALESCE(e.POLICY_NUMBER,t.POLICY_NUMBER) AS POLICY_NUMBER,  
        CASE  
            WHEN e.POLICY_NUMBER IS NULL THEN 'EXTRA_ROW_IN_TARGET'  
            WHEN t.POLICY_NUMBER IS NULL THEN 'MISSING_ROW_IN_TARGET'  
            WHEN e.RowHash <> t.RowHash THEN 'DATA_MISMATCH'  
        END AS Validation_Result  
    FROM ExpectedHash e
```

Validation

```
FULL OUTER JOIN TargetHash t
ON e.POLICY_NUMBER = t.POLICY_NUMBER
WHERE
e.POLICY_NUMBER IS NULL
OR t.POLICY_NUMBER IS NULL
OR e.RowHash <> t.RowHash
) V;

IF EXISTS (SELECT 1 FROM #HashValidation)
BEGIN
    SELECT *
    FROM #HashValidation;
END
ELSE
BEGIN
    SELECT
    'SUCCESS' AS Validation_Status,
    'All records successfully passed hash validation. Target matches the expected transformed dataset.' AS Message;
END;

DROP TABLE #HashValidation;
```



```
1 WITH cte_dedup AS
2 (
3     SELECT *,
4     ROW_NUMBER() OVER
5     FROM #HashValidation
6 )
7 SELECT
8     'SUCCESS' AS Validation_Status,
9     'All records successfully passed hash validation. Target matches the expected transformed dataset.' AS Message;
10
```

Results (1) Messages

Result Set Batch 1 - Query 1

Validation_Status	Message
SUCCESS	All records successfully passed hash validation. Target matches the expected transformed dataset.

(1 row affected)

Validation

Expected vs. Actual Validation

Performed Expected vs. Actual validation by recomputing the expected target dataset from the source using the STTM business rules, including deduplication, cleansing, lookups, and transformations. The recomputed dataset was compared with the target table to identify missing, extra, or mismatched records, ensuring the ETL output accurately reflects the defined business rules.

WITH cte_dedup AS

```
(
    SELECT *,
    ROW_NUMBER() OVER
    (
        PARTITION BY POL_NBR
        ORDER BY SRC_UPDATE_TS DESC,
        COALESCE(PREM_AMT,0) DESC
    ) rn
    FROM dbo.STG_POLICY
),
```

Expected AS

```
(
    SELECT
    TRIM(d.POL_NBR) AS POLICY_NUMBER,
    TRY_CONVERT(DATE,NULLIF(TRIM(d.EFF_DT),"),112) AS EFFECTIVE_DATE,
    CAST(COALESCE(d.PREM_AMT,0.00) AS DECIMAL(14,2)) AS PREMIUM_AMOUNT,
    rs.STATE_CD AS STATE_CODE,
    COALESCE(rl.LOB_DESC,'UNKNOWN') AS LINE_OF_BUSINESS,
    CONCAT_WS(' ',
        NULLIF(TRIM(d.LNAME),"),
        NULLIF(TRIM(d.FNAME),")
    ) AS INSURED_NAME,
    CASE UPPER(TRIM(d.CANCEL_FLG))
    WHEN 'Y' THEN 'CANCELLED'
    WHEN 'N' THEN 'ACTIVE'
    ELSE 'UNKNOWN'
```

Validation

```
END AS STATUS
FROM cte_dedup d
LEFT JOIN dbo.REF_STATE rs
ON UPPER(TRIM(d.ST_CD)) = rs.STATE_CD
LEFT JOIN dbo.REF_LOB rl
ON TRIM(d.LOB_CD) = rl.LOB_CD
WHERE rn = 1
AND TRY_CONVERT(DATE, NULLIF(TRIM(d.EFF_DT), '')) IS NOT NULL
AND rs.STATE_CD IS NOT NULL
)
```

```
SELECT *
FROM Expected
```

```
EXCEPT
```

```
SELECT
    POLICY_NUMBER,
    EFFECTIVE_DATE,
    PREMIUM_AMOUNT,
    STATE_CODE,
    LINE_OF_BUSINESS,
    INSURED_NAME,
    STATUS
FROM dbo.ILM_POLICY
```

```
UNION ALL
```

```
SELECT
    POLICY_NUMBER,
    EFFECTIVE_DATE,
    PREMIUM_AMOUNT,
```

Validation

```
STATE_CODE,  
LINE_OF_BUSINESS,  
INSURED_NAME,  
STATUS  
FROM dbo.ILM_POLICY
```

EXCEPT

```
SELECT *  
FROM Expected;
```

65
66
67
68
69
70

FROM dbo.ILM_POLICY

EXCEPT

SELECT *
FROM Expected;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE

Results (1) Messages

Result Set Batch 1 - Query 1

=====

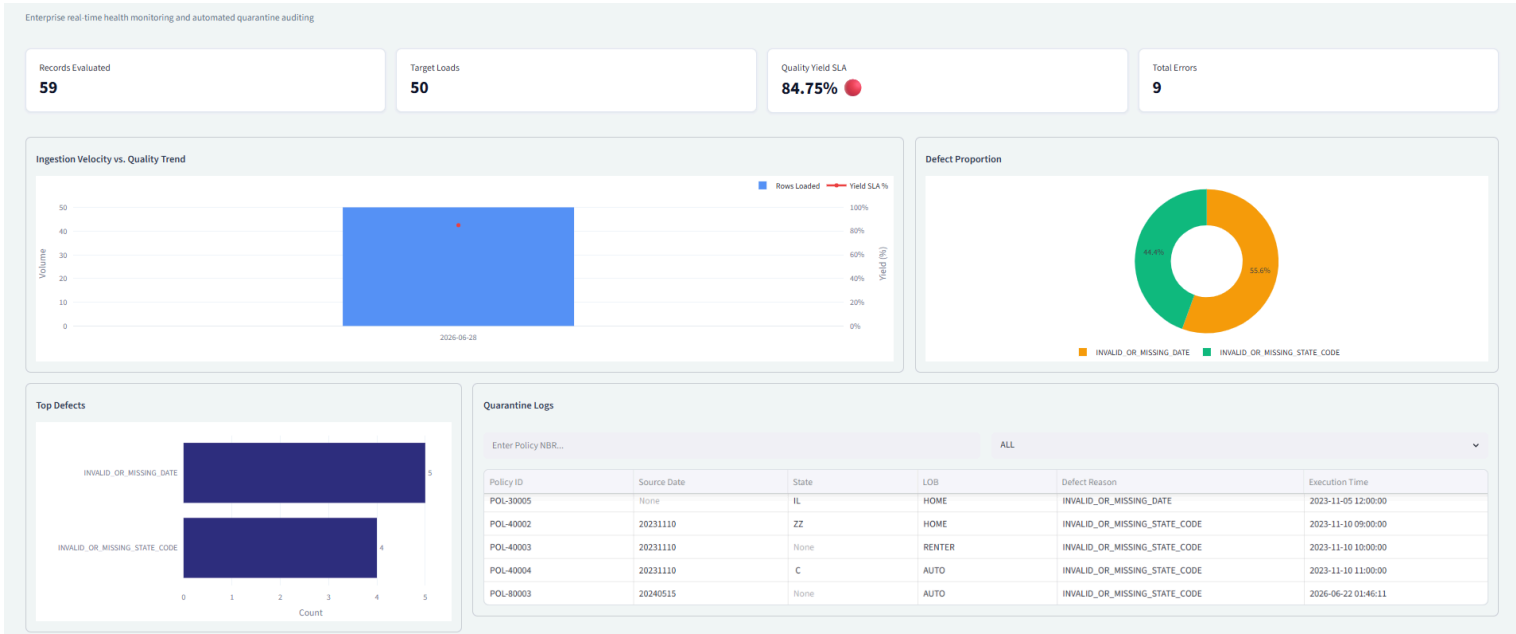
POLICY_NUMBER	EFFECTIVE_DATE	PREMIUM_AMOUNT	STATE_CODE	LINE_OF_BUSINESS	INSURED_NAME	STATUS

(0 rows affected)

Dashboard

Dashboard

Initial Insights



Code:

```
import streamlit as st

import pandas as pd

import pyodbc

import plotly.graph_objects as go

import plotly.express as px

from plotly.subplots import make_subplots


# =====

# 1. PAGE CONFIG

# =====

st.set_page_config(
    page_title="ETL Command Center",
    page_icon="⚙️",
    layout="wide",
    initial_sidebar_state="collapsed"
)


# =====

# =====
```

Dashboard

```
st.markdown("""  
<style>  
  
/* 1. Set the entire dashboard background to a soft BI-style gray */  
[data-testid="stAppViewContainer"] {  
    background-color: #F1F5F9;  
}  
  
/* 2. Tighten the main canvas padding */  
.block-container {  
    padding-top: 1.5rem !important;  
    padding-bottom: 1.5rem !important;  
    padding-left: 2rem !important;  
    padding-right: 2rem !important;  
    max-width: 100% !important;  
}  
  
/* 3. Style the KPI Cards to look like solid white BI tiles */  
div[data-testid="stMetric"] {  
    background-color: #FFFFFF;  
    border: 1px solid #E2E8F0;  
    border-radius: 8px;  
    padding: 15px 20px;  
    box-shadow: 0 1px 3px rgba(0,0,0,0.05);  
}  
  
div[data-testid="stMetricValue"] {  
    font-size: 26px !important;  
    font-weight: 700 !important;  
    color: #0F172A !important;  
}  
  
div[data-testid="stMetricLabel"] {  
    font-size: 13px !important;  
    font-weight: 600 !important;  
    color: #64748B !important;  
    text-transform: uppercase;
```

Dashboard

```
letter-spacing: 0.5px;
}
```

/* 4. Online Status Pill */

```
.status-pill {
  background-color: #DCFCE7;
  color: #166534;
  padding: 6px 14px;
  border-radius: 12px;
  font-size: 12px;
  font-weight: 700;
  border: 1px solid #BBF7D0;
}
```

</style>

""", unsafe_allow_html=True)

=====

3. HEADER ARCHITECTURE

=====

title_col, status_col = st.columns([6, 1])

with title_col:

```
st.markdown("<h2 style='margin-bottom: 0px; color: #0F172A;'>ETL Quality Command Center</h2>", unsafe_allow_html=True)
st.markdown("<p style='color: #64748B; font-size: 14px; margin-top: -5px;'>Enterprise real-time health monitoring and automated quarantine auditing</p>", unsafe_allow_html=True)
```

with status_col:

```
st.markdown('<div style="text-align:right; padding-top: 10px;"><span class="status-pill">● SYSTEM ONLINE</span></div>',
unsafe_allow_html=True)
```

st.markdown("<div style='margin-bottom: 20px;'></div>", unsafe_allow_html=True)

=====

4. DATABASE ACQUISITION LAYER

=====

Dashboard

@st.cache_data(ttl=15)

def fetch_pipeline_data():

 conn_str = (

 r"DRIVER={ODBC Driver 17 for SQL Server};"

 r"SERVER=<Server name>,"

 r"DATABASE=<Database>,"

 r"Trusted_Connection=yes;"

)

 conn = pyodbc.connect(conn_str)

 timeline_query = """

 WITH SuccessTimeline AS (

 SELECT CAST(LOAD_TS AS DATE) as LogDate, COUNT(*) as RowsLoaded

 FROM dbo.ILM_POLICY WHERE LOAD_TS IS NOT NULL GROUP BY CAST(LOAD_TS AS DATE)

),

 ErrorTimeline AS (

 SELECT CAST(GETDATE() AS DATE) as LogDate, COUNT(*) as TotalErrors

 FROM dbo.ERROR_POLICY

),

 AllDates AS (

 SELECT LogDate FROM SuccessTimeline UNION SELECT LogDate FROM ErrorTimeline

)

 SELECT

 d.LogDate,

 COALESCE(s.RowsLoaded, 0) as RowsLoaded,

 COALESCE(e.TotalErrors, 0) as TotalErrors,

 CASE

 WHEN (COALESCE(s.RowsLoaded, 0) + COALESCE(e.TotalErrors, 0)) > 0

 THEN CAST(COALESCE(s.RowsLoaded, 0) AS FLOAT) / (COALESCE(s.RowsLoaded, 0) + COALESCE(e.TotalErrors, 0))

 ELSE 1.0

 END as YieldRate

 FROM AllDates d

 LEFT JOIN SuccessTimeline s ON d.LogDate = s.LogDate

 LEFT JOIN ErrorTimeline e ON d.LogDate = e.LogDate;

Dashboard

"""

```
error_profile_query = """SELECT REJECT_REASON, COUNT(*) as ErrorCount FROM dbo.ERROR_POLICY GROUP BY REJECT_REASON;"""
```

```
quarantine_query = """SELECT POL_NBR, EFF_DT, ST_CD, LOB_CD, REJECT_REASON, SRC_UPDATE_TS FROM dbo.ERROR_POLICY;"""
```

```
df_time = pd.read_sql(timeline_query, conn)
```

```
df_err = pd.read_sql(error_profile_query, conn)
```

```
df_quar = pd.read_sql(quarantine_query, conn)
```

```
conn.close()
```

```
if not df_time.empty:
```

```
    df_time['LogDate'] = df_time['LogDate'].astype(str)
```

```
    df_time = df_time.sort_values('LogDate')
```

```
return df_time, df_err, df_quar
```

```
try:
```

```
    df_time, df_err, df_quar = fetch_pipeline_data()
```

```
except Exception as database_exception:
```

```
    st.error(f"Database Connectivity Failure: {database_exception}")
```

```
    st.stop()
```

```
# =====
```

```
# 5. KPI SCORECARD (Top Row)
```

```
# =====
```

```
total_success = int(df_time['RowsLoaded'].sum()) if not df_time.empty else 0
```

```
total_errors = int(df_time['TotalErrors'].sum()) if not df_time.empty else 0
```

```
total_processed = total_success + total_errors
```

```
global_yield = (total_success / total_processed) * 100 if total_processed > 0 else 0.0
```

```
kpi1, kpi2, kpi3, kpi4 = st.columns(4)
```

```
with kpi1:
```

```
    st.metric("Records Evaluated", f"{total_processed:,}")
```

Dashboard

with kpi2:

```
st.metric("Target Loads", f"{total_success:,}")
```

with kpi3:

```
status_indicator = "🟢" if global_yield >= 95 else "🟡" if global_yield >= 85 else "🔴"
```

```
st.metric("Quality Yield SLA", f"{global_yield:.2f}% {status_indicator}")
```

with kpi4:

```
st.metric("Total Errors", f"{total_errors:,}")
```

```
st.markdown("<div style='margin-bottom: 20px;'></div>", unsafe_allow_html=True)
```

```
# =====
```

```
# 6. MIDDLE GRID: TIMELINE & PIE CHART
```

```
# =====
```

```
col_timeline, col_pie = st.columns([6, 4])
```

with col_timeline:

```
# Wrapping inside a bordered container creates the "BI Card" look
```

```
with st.container(border=True):
```

```
st.markdown("<h4 style='font-size: 15px; color: #334155; margin-bottom: 0px;'>Ingestion Velocity vs. Quality Trend</h4>", unsafe_allow_html=True)
```

```
fig_timeline = make_subplots(specs=[[{"secondary_y": True}]])
```

```
fig_timeline.add_trace(
```

```
    go.Bar(x=df_time['LogDate'], y=df_time['RowsLoaded'], name="Rows Loaded", marker_color='#3B82F6', opacity=0.85), secondary_y=False
```

```
)
```

```
fig_timeline.add_trace(
```

```
    go.Scatter(x=df_time['LogDate'], y=df_time['YieldRate'] * 100, name="Yield SLA %", line=dict(color='#EF4444', width=3), mode='lines+markers'), secondary_y=True
```

```
)
```

```
# Reduced height to 280 to stop it from looking "too huge"
```

```
fig_timeline.update_layout(
```

```
    plot_bgcolor='white', paper_bgcolor='white',
```

```
    bargap=0.7 if len(df_time) == 1 else 0.2, # Keeps single bar thin
```

Dashboard

```
axis=dict(title="", type="category", showgrid=False),
yaxis=dict(title="Volume", showgrid=True, gridcolor='#F1F5F9'),
yaxis2=dict(title="Yield (%)", ticksuffix="%", range=[0, 105], showgrid=False, overlaying="y", side="right"),
legend=dict(orientation="h", yanchor="bottom", y=1.05, xanchor="right", x=1),
margin=dict(l=0, r=0, t=20, b=0),
height=280
)

st.plotly_chart(fig_timeline, use_container_width=True)
```

with col_pie:

```
with st.container(border=True):

    st.markdown("<h4 style='font-size: 15px; color: #334155; margin-bottom: 0px;'>Defect Proportion</h4>",
unsafe_allow_html=True)

    if not df_err.empty:

        fig_pie = px.pie(
            df_err, values='ErrorCount', names='REJECT_REASON', hole=0.5,
            color_discrete_sequence=['#F59E0B', '#10B981', '#3B82F6', '#6366F1', '#EC4899']
        )

        fig_pie.update_layout(
            plot_bgcolor='white', paper_bgcolor='white',
            margin=dict(l=0, r=0, t=20, b=0), height=280,
            legend=dict(orientation="h", yanchor="bottom", y=-0.2, xanchor="center", x=0.5)
        )

        st.plotly_chart(fig_pie, use_container_width=True)

    else:

        st.info("No defects logged.")
```

```
# =====
```

```
# 7. BOTTOM GRID: BAR CHART & QUARANTINE LOGS
```

```
# =====
```

```
col_bar, col_table = st.columns([3, 7])
```

with col_bar:

```
with st.container(border=True):

    st.markdown("<h4 style='font-size: 15px; color: #334155; margin-bottom: 0px;'>Top Defects</h4>", unsafe_allow_html=True)
```

Dashboard

```
if not df_err.empty:
    df_err_sorted = df_err.sort_values(by='ErrorCount', ascending=True)
    fig_bar = px.bar(
        df_err_sorted, x='ErrorCount', y='REJECT_REASON', orientation='h',
        text='ErrorCount', color_discrete_sequence=['#312E81']
    )
    fig_bar.update_traces(textposition='outside')
    fig_bar.update_layout(
        plot_bgcolor='white', paper_bgcolor='white',
        xaxis=dict(title="Count", showgrid=True, gridcolor='#F1F5F9'),
        yaxis=dict(title="", showgrid=False),
        margin=dict(l=0, r=20, t=20, b=0), height=300
    )
    st.plotly_chart(fig_bar, use_container_width=True)
else:
    st.info("No defects logged.")

with col_table:
    with st.container(border=True):
        st.markdown("<h4 style='font-size: 15px; color: #334155; margin-bottom: 15px;'>Quarantine Logs</h4>",
            unsafe_allow_html=True)

    if not df_quar.empty:
        search_col, reason_col = st.columns([1, 1])

        with search_col:
            search_term = st.text_input("Search ID:", placeholder="Enter Policy NBR...", label_visibility="collapsed")

        with reason_col:
            distinct_reasons = ["ALL"] + list(df_quar['REJECT_REASON'].unique())
            selected_reason = st.selectbox("Filter:", distinct_reasons, label_visibility="collapsed")

        filtered_df = df_quar.copy()

        if search_term:
            filtered_df = filtered_df[filtered_df['POL_NBR'].astype(str).str.contains(search_term, case=False, na=False)]

        if selected_reason != "ALL":
            filtered_df = filtered_df[filtered_df['REJECT_REASON'] == selected_reason]
```

Dashboard

```
st.dataframe(
    filtered_df,
    column_config={
        "POL_NBR": st.column_config.TextColumn("Policy ID"),
        "EFF_DT": st.column_config.TextColumn("Source Date"),
        "ST_CD": st.column_config.TextColumn("State"),
        "LOB_CD": st.column_config.TextColumn("LOB"),
        "REJECT_REASON": st.column_config.TextColumn("Defect Reason"),
        "SRC_UPDATE_TS": st.column_config.DatetimeColumn("Execution Time")
    },
    use_container_width=True, hide_index=True, height=205 # Constrained table height
)
else:
    st.success("✅ Quarantine table is empty.")
```